

Práctica:

Expresiones regulares aplicadas en Bash y Python

Teoría Matemática de la Computación

Dr. José Luis Quiroz Fabián

1. Objetivo

Implementar validadores y analizadores de texto utilizando expresiones regulares en Bash y Python, aplicados al análisis de archivos de log y validación de contraseñas, con el propósito de modelar escenarios reales del ámbito profesional en DevOps y desarrollo de software.

2. Entrega

Se deberá crear un repositorio en su organización con el siguiente nombre:

`regex-lab`

2.1. Estructura obligatoria del repositorio

```
regex-lab/  
  README.md  
  data/  
    log_muestra_app.log  
    passwords_muestra.txt  
  src/  
    log_reporter_grep.sh  
    log_reporter_re.py  
    password_validator_grep.sh  
    password_validator_re.py  
  out/
```

- Los scripts deben ejecutarse desde la raíz del repositorio.
- La carpeta `out/` debe generarse automáticamente si no existe.
- El archivo `README.md` debe explicar cómo ejecutar cada script.

3. Ejercicio 1: análisis de logs del sistema

3.1. Contexto

Los sistemas reales generan archivos de log para auditoría y monitoreo. Es necesario detectar automáticamente errores válidos y generar reportes.

3.2. Archivo de entrada

`data/log_muestra_app.log`

3.3. Formato válido de línea

Una línea se considera válida si cumple:

`[NIVEL] AAAA-MM-DD HH:MM:SS Mensaje`

donde NIVEL pertenece a:

`INFO | WARN | ERROR | DEBUG`

El mensaje debe contener al menos un carácter.

3.4. Script 1: Bash

Archivo requerido:

`src/log_reporter_grep.sh`

Debe:

1. El usuario debe especificar el nivel que desea extraer INFO, WARN, ERROR o DEBUG.
2. Leer el archivo de log.
3. Extraer únicamente líneas válidas (en formato) con el nivel especificado.
4. Guardarlas en:

`out/xxxx_validos.txt`, donde xxxx es info, warn, error o debug según el nivel

5. Mostrar en pantalla:

- total de líneas no vacías,
- total de líneas válidas,
- total de líneas sospechosas (contienen el nivel especificado pero no cumplen el formato).

6. Generar:

`out/reporte_log.json`

3.5. Script 2: Python

Archivo requerido:

`src/log_reporter_re.py`

Debe realizar exactamente lo mismo que el script en Bash, pero utilizando el módulo `re`.

4. Ejercicio 2: validación de contraseñas

4.1. Archivo de entrada

`data/passwords_muestra.txt`

4.2. Reglas del lenguaje

Una contraseña es válida si:

- Tiene longitud mayor o igual a 8.
- Contiene al menos una letra mayúscula.
- Contiene al menos un dígito.
- Solo contiene letras y números.

4.3. Script 3: Bash

Archivo requerido:

`src/password_validator_grep.sh`

Debe:

1. Leer el archivo de contraseñas.
2. Separar y guardar:
 - válidas en `out/validas.txt`
 - inválidas en `out/invalidas.txt`
3. Mostrar el total de válidas e inválidas.
4. Mostrar las razones de rechazo:
 - longitud insuficiente,
 - no tiene mayúscula,
 - no tiene dígito,
 - tiene caracteres inválidos.

4.4. Script 4: Python

Archivo requerido:

```
src/password_validator_re.py
```

Debe:

1. Implementar una función `is_valid(password)`.
2. Debe realizar exactamente lo mismo que el script en Bash, pero utilizando el módulo `re`.

5. Criterios de evaluación

- Exactitud de las expresiones regulares.
- Funcionamiento.
- Estructura adecuada del repositorio.
- Claridad en README.
- Manejo básico de errores.

6. Reflexión final

Explique brevemente:

- ¿Por qué conviene validar datos tanto en frontend como en backend?
- ¿Qué limitaciones tienen las expresiones regulares en sistemas reales?